CQA HACKATHON · IDEA PLAYBOOK

Ideas, compressed

An 8-part foundation you must ship, plus 17 upgrades. Every card answers four questions: what is it, why does it work, what does it cost, and what do we show the jury.

FOUNDATION · ship these first

F1 Checklist-as-Data

Versioned JSON. Prompts are compiled from it, so a new criterion needs zero code.

F2 LLM judges, code counts

The model returns verdicts; plain code does weights, N/A redistribution and rounding.

F3 Evidence-locked verdicts

Every verdict cites message IDs, quotes are verified, and a click jumps to the message or audio second.

F4 Signals layer

Facts computed before the AI sees the chat: waits, languages, system events, risk words.

F5 Original-language judging

Never translate first. Test two models on the 15 Kazakh chats before choosing.

F6 Appeals workflow

Operator and Supervisor roles, clear states, AI original → decision → final, audit log.

F7 Control-round pipeline

Parallel calls, cache, re-run only the changed criterion, before/after diff.

F8 Dynamics-ready UI

Vanilla JS Web Components + one API adapter → an HTML web resource.

WHAT THE JURY WILL CHECK · the 5-minute demo

0:00–1:00
Interface + one dialog
open a Kazakh chat from the list

1:00–3:00
Evaluation with receipts
evidence jump · an N/A example · the total and how it was built

3:00–4:00
Appeal
operator disputes → supervisor decides → new total

4:00–5:00
Criterion change
swap a criterion in the UI → re-run → diff

Hidden tests: Replace one criterion through the UI · Control transcripts arrive at hour 23

Both are covered by F1, F7 and upgrade 6.

HOW TO READ THE UPGRADE CARDS

Each card has a colour for its theme, a number you can quote in the team chat, and four lines: **What** it is, **Why** it works (with the research or fact behind it), what it costs to **Build** for one developer, and the **Demo** moment it gives you.

Accuracy	1–3
Trust	4–5
Learning	6–7
Kazakhstan & broker	8–11
Voice	12
Beyond the score	13–15
Integrate & scale	16–17

Upgrades 1–9

Accuracy, trust and learning: make the judge more correct, measure it, and let it improve.

1 Atomic sub-questions

ACCURACY

What Split each criterion into 2–5 yes/no atoms; a decision table turns them into the verdict.

Why CheckEval: +0.45 agreement across 12 judge models, lower variance.

Build atoms live in the checklist JSON, AI-drafted on save · ~3 h

Demo expand a criterion → every atom with its own evidence.

2 Extract → Judge

ACCURACY

What Pass 1 quotes the evidence per atom; pass 2 decides only from those quotes.

Why Quote-first prompting cuts hallucination; "none in m1–m9" becomes the checked range.

Build two calls or two output sections, cached prefix · ~2 h

Demo a "Show AI notes" toggle under each verdict.

3 Jury + arbiter

ACCURACY

What Two judges from different model families; a third model decides only when they disagree.

Why Panels of diverse models beat one big judge, with less bias and about 7× lower cost.

Build one client for two providers, "2/2 agree" badge · ~2–3 h

Demo a split verdict resolved live by the arbiter.

4 Trust-or-Escalate

TRUST

What Calibrate a confidence threshold on labelled DEV verdicts; auto-accept above it, queue the rest.

Why ICLR 2025: over 80% human agreement guaranteed at about 80% coverage, even with a 7B judge.

Build confidence = jury agreement; Wilson lower bound · ~3 h

Demo a slider: target agreement → share auto-accepted.

5 Judge red team

TRUST

What Break dialogs on purpose (drop the greeting, add a fraud offer); the verdict must flip. Harmless edits must not.

Why Evaluator LLMs missed quality drops in over half of cases (EMNLP 2024).

Build mutation library + runner = a free regression suite · ~2–3 h

Demo a robustness report, e.g. "caught 47 of 50".

6 Criterion Studio

LEARNING

What AI drafts a new criterion (definition, N/A rule, atoms, tests); you grade 5 dialogs; a linter flags ambiguity.

Why People refine criteria while grading ("criteria drift", UIST 2024).

Build pick the 5 most uncertain dialogs + lint rules · ~4 h

Demo the jury's criterion swap in about a minute.

7 Appeal-driven evolution

LEARNING

What Reviewer reasons on overturned appeals → an optimizer rewrites the criterion → tests → human approval.

Why GEPA (ICLR 2026): reflective prompt evolution beats RL with up to 35× fewer rollouts.

Build mini version: one rewrite call + validation · ~2–3 h

Demo a checklist diff with before / after agreement.

8 Code-switching engine

KAZAKHSTAN & BROKER

What Per-message language ID, a Kazakh–Russian mix profile, and "client's language" as a policy setting.

Why Mixing is everyday, even inside one word ("magazinge").

Build 9 Kazakh-only letters + stopwords, LLM fallback · ~2 h

Demo a language chip: "KZ 70% · RU 30%".

9 Privacy shield

KAZAKHSTAN & BROKER

What Typed, reversible masking ([IIN_1], [CARD_1], [NAME_1]) before any AI call.

Why Typed tokens keep criteria 03 and 08 working; generic tools need Kazakh recognizers.

Build regex for IIN, +7 7xx phones, KZ IBAN, Luhn-checked cards · ~2 h

Demo a "What the AI saw" toggle.

Upgrades 10–17

Local fit, voice, value beyond the score, and how it scales.

10 Compliance-by-design

KAZAKHSTAN & BROKER

What AI proposes, humans decide: AI labels, version + prompt hash, appeal everywhere, audit export.

Why Kazakhstan's AI Law (in force 18 Jan 2026): transparency, explainability, human oversight.

Build mostly making existing metadata visible · ~1 h

Demo one screen and one sentence in the pitch.

11 Risk radar

KAZAKHSTAN & BROKER

What A gross error opens a compliance case: alert, evidence pack, per-operator counter, near-miss watchlist.

Why At a broker, an off-platform money offer is fraud risk; the offer alone counts.

Build broad scan → jury confirmation → webhook · ~2 h

Demo a red alert pointing at the exact message.

12 Audio-native signals

VOICE

What Send ±15 s audio windows to an audio model: interruptions, irritation, unexplained silence.

Why Many tone and voice cues simply are not in a transcript.

Build ffmpeg windows + yes/no questions + talk/silence ratio · ~3–4 h

Demo an "experimental audio signals" panel on a call.

13 Coaching loop

BEYOND THE SCORE

What A "golden reply" for every lost point, plus "Practice this" role-play scored by the same engine.

Why QA that only punishes rarely changes behaviour; AI role-play training is spreading fast.

Build golden reply ~1 h · role-play ~3 h (reuses the engine)

Demo click "Practice this" on a failed criterion.

14 Contact-reason insights

BEYOND THE SCORE

What An auto-built map of why clients write, crossed with where operators fail.

Why TnT-LLM (Microsoft): LLM-made taxonomies and labels at scale; keep a human editor.

Build one-line reason per dialog → categories → heatmap · ~2 h

Demo a heatmap of contact reasons × criteria.

15 Explainable score UI

BEYOND THE SCORE

What A points waterfall after N/A redistribution, what-if toggles, and an evidence timeline strip.

Why Makes the N/A math visible at a glance and shows whether an appeal is worth it.

Build pure frontend on top of the scoring function · ~2 h

Demo flip a verdict → the total updates live.

16 CQA as an MCP server

INTEGRATE & SCALE

What Tools evaluate / get_result / file_appeal / query_scores, usable by any AI assistant.

Why Dynamics' Dataverse already exposes an MCP server (GA since Ignite 2025).

Build official MCP SDK, four tools · ~1–2 h

Demo ask an assistant "who failed identification in KZ chats?"

17 Cost cascade

INTEGRATE & SCALE

What Cheap judge first, strong model only when unsure, cached prefix, live $ per evaluation.

Why Cascades keep agreement at lower cost; batch APIs halve the price but can take up to 24 h.

Build log token usage on every call · ~1 h

Demo "≈ $ per evaluation" on every card.

Where to go next

Page 4 says what to build in the 24 hours and what to pitch as roadmap.

The **Architecture** PDF shows where each idea lives: the pipeline (1–5, 8, 9, 11, 12, 17), the checklist service and quality loops (6, 7), the appeal flow and audit (4, 10), the UI (13–15) and the API layer (16).

Effort figures are rough hours for one developer who already has the foundation.

What to build in 24 hours

Ship the foundation, add the upgrades that make a demo step stronger, and pitch the rest as a roadmap backed by research.

BUILD NOW

- F1–F8 Foundation
- 1 Atomic sub-questions
- 2 Extract → Judge
- 5 Judge red team
- 6 Criterion Studio
- 9 Privacy shield
- 15 Explainable score UI

Each one powers a step of the 5-minute demo.

IF AHEAD

- 3 + 4 Jury and trust slider
- 13 Golden replies (skip role-play)
- 11 Risk radar
- 8 Language engine

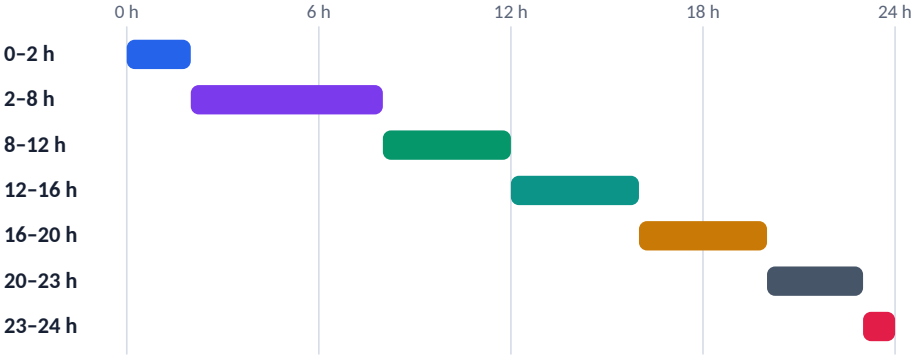
Add in this order; stop the moment the demo is at risk.

PITCH AS ROADMAP

- 7 Appeal-driven evolution
- 12 Audio-native signals
- 14 Contact-reason insights
- 16 MCP server + live co-pilot
- 10 AI-law framing (just say it)

One slide. Each item is backed by a paper or a shipped product.

A 24-HOUR TIMELINE



- **0–2 h** — Data import · normalizer · checklist JSON · scoring engine + unit tests
- **2–8 h** — Judge engine (atoms, extract → judge) · validator · evaluation card with evidence jump
- **8–12 h** — Appeals · checklist editor · batch runner with cache + incremental re-run
- **12–16 h** — Gold-set labels · red-team suite · Criterion Studio
- **16–20 h** — Privacy shield · waterfall + what-if · README · polish
- **20–23 h** — Rehearse the demo and the control flow with a stopwatch · feature freeze
- **23–24 h** — Process the control set · verify · final commit before the stop-code

THE PITCH IN THREE VERBS

Explain · Measure · Improve

Every score has receipts. We know when to trust the AI. Every appeal makes it better.

LIKELY JURY QUESTIONS

What does one evaluation cost? Logged on every call and shown on every card, with the model name.

How do you know the AI is right? Gold-set agreement per criterion, a red-team score, and low-confidence verdicts go to people.

What happens when a criterion changes? New checklist version; only that criterion re-runs; old scores keep their version.

Sources behind the ideas

Worth a look before the pitch. Check current model prices on the providers' pricing pages; they change often.

1 **CheckEval: a reliable LLM-as-a-Judge framework using checklists**
Lee et al. · EMNLP 2025 · arXiv 2403.18771

2 **Anthropic docs: long context prompting tips (quote first)**
platform.claude.com/docs · prompt engineering

3 **Replacing Judges with Juries (PoLL)**
Verga et al., Cohere · 2024 · arXiv 2404.18796

3 **DAFE: two judges plus an arbiter on disagreement**
arXiv, LLM-based evaluation

4 **Trust or Escalate: LLM judges with provable guarantees**
Jung, Brahman, Choi · ICLR 2025 · arXiv 2407.18370

5 **Finding Blind Spots in Evaluator LLMs (FBI)**
Doddapaneni et al. · EMNLP 2024 · arXiv 2406.13439

6 **Who Validates the Validators? (EvalGen, criteria drift)**
Shankar et al. · UIST 2024 · arXiv 2404.12272

7 **GEPA: Reflective Prompt Evolution Can Outperform RL**
Agrawal et al. · ICLR 2026 · arXiv 2507.19457 · dspy.GEPA

8 **ISSAI Kazakh Speech Corpus 2 (code-switching)**
Mussakhojayeva et al. · Interspeech 2022

8 **Bilingual model for Kazakh–Russian code-switching ASR**
Ubskii et al. · CEUR-WS Vol-2590

9 **Microsoft Presidio: PII detection and pseudonymization**
open source; needs custom recognizers beyond English

10 **Law of the Republic of Kazakhstan No. 230-VIII "On Artificial Intelligence"**
signed 17 Nov 2025 · in force 18 Jan 2026

12 **Qwen3-Omni / Qwen3-Omni-Captioner; open audio decision models**
Hugging Face model cards

13 **LLM role-play training with an adaptive virtual customer**
Frontiers in AI · 2026

14 **TnT-LLM: text mining at scale with LLMs**
Wan et al., Microsoft · KDD 2024 · arXiv 2403.12173

16 **Dataverse as an MCP server; HTML web resources**
Microsoft Learn (Power Apps docs)

17 **Batch processing and prompt caching**
provider docs; batch ≈ 50% off, results within 24 h

Numbers on the left are the upgrade cards each source supports.